

Next.js, the mental model

How routing, components, rendering, data, caching and streaming fit together into one story.



Routing



Components



Rendering



Data fetching



Caching



User experience

One story, four questions



Where does this happen?

Server or browser?



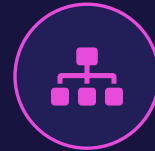
When does this happen?

Build time, request time, or later?



Why does Next.js do it this way?

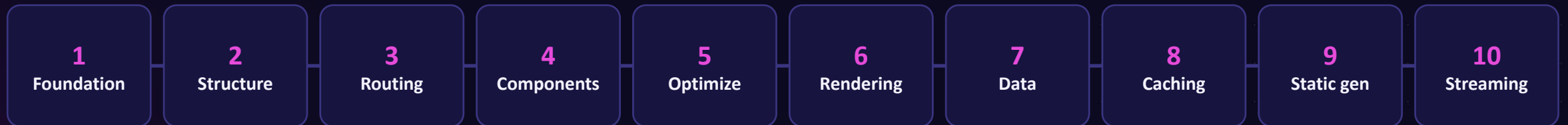
What problem does it solve?



How does it connect?

What comes before and after it?

OUR JOURNEY



Routing → Component → Rendering → Data fetching → Caching → User experience

React gives you the UI. Next.js gives you the app.

React alone

A library. You choose, install and wire every piece.

Routing

you decide

Server rendering

you decide

Bundling & code splitting

you decide

Image handling

you decide

Data loading pattern

you decide

SEO & metadata

you decide



Next.js

A framework on top of React, with defaults built in.

Optimizations image, font, script, splitting

Data fetching & caching on the server, by default

Rendering server, static or client, per page

Routing & layouts from your folders

React components, JSX, state



Next.js does not replace React. Your components are still React components.

Why choose Next.js? Three ideas



Convention over configuration

Put a file in the right place and it just works.
No routing config to write.

```
app/about/page.tsx  
→ /about
```

Why? Less setup, more building.



Server-first

Do the heavy work on the server. Send the browser finished HTML and less JavaScript.

```
Server Component  
is the default
```

Why? Faster loads, safer secrets.



Fast by default

Images, fonts, scripts and code splitting are handled for you.

```
<Image src=... />  
optimized for you
```

Why? Speed without extra effort.

A Next.js project, file by file

```
my-app/  
├── app/  
│   ├── layout.tsx  
│   ├── page.tsx  
│   ├── globals.css  
│   └── products/  
│       └── [id]/  
│           └── page.tsx  
├── components/  
│   └── ProductCard.tsx  
├── public/  
│   └── logo.png  
└── next.config.ts
```



app/

Routes live here. Folders become URLs.



page.tsx

The unique UI of one route.



layout.tsx

Shared UI that wraps pages.



components/

Reusable pieces. Not routes.



public/

Static files, served from the root.

Folder → URL. *layout.tsx wraps page.tsx. Colors match the tree.*

File-system routing: folders are your URLs

FILE

`app/page.tsx`

`app/about/page.tsx`

`app/blog/page.tsx`

`app/blog/first-post/page.tsx`

URL

`/`

`/about`

`/blog`

`/blog/first-post`



A folder becomes a route only when it contains a page.tsx. No routing config to maintain.

Link and client-side navigation

`<a href>`

Full page reload

Click

Ask server for the full page

Reload everything

Screen flashes, state lost

`<Link>`

Client-side navigation

Click

Fetch only what is new

Swap the page area

Layout stays, no reload

```
<Link href="/about">About</Link>
```



Link also prefetches routes that are visible on screen, so the next page is often ready before the click.

Dynamic routes: one file, many URLs



```
export default async function Page({ params }) {  
  const { id } = await params;  
  return <h1>Product {id}</h1>;  
}
```

Nested routes and nested layouts

`app/layout.tsx`

Root layout

`app/dashboard/layout.tsx`

Dashboard layout (sidebar)

`app/dashboard/settings/page.tsx`

The page: unique to `/dashboard/settings`



`/dashboard/settings`

Root layout

Wraps every route. Holds html, body and shared navbar.

Dashboard layout

Wraps only routes inside `/dashboard`.

Page

The innermost piece, unique to the URL.



Navigate to `/dashboard/profile`: layouts stay and keep their state. Only the page swaps.

Server Components vs Client Components



Server Component

Default. Runs on the server.

- ✓ Fetch data directly
- ✓ Use databases and secrets safely
- ✓ Sends HTML, not its own JavaScript
- ✗ No useState, useEffect or onClick



Client Component

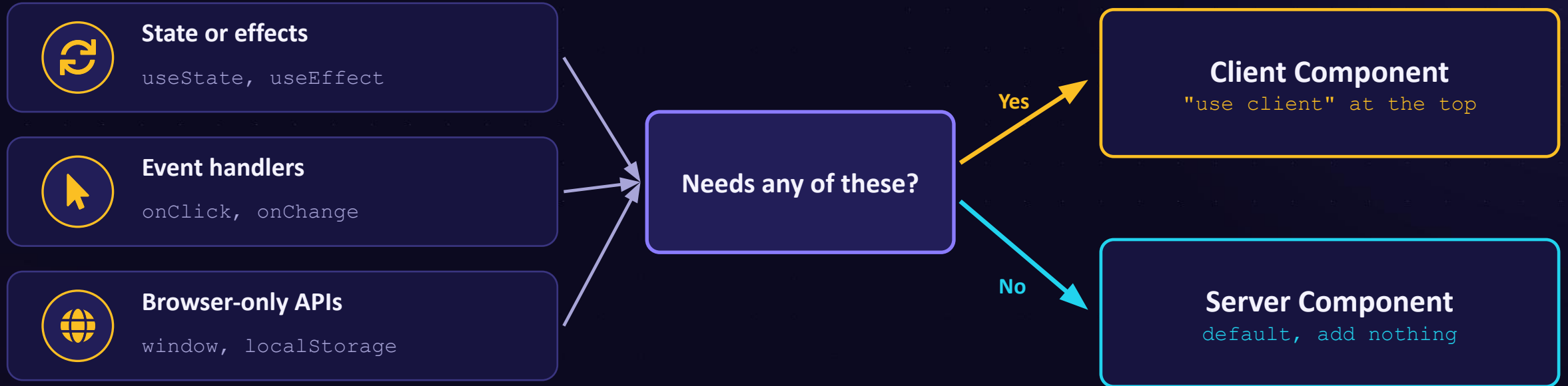
Opt in with "use client".

- ✓ useState, useEffect and other hooks
- ✓ onClick, onChange and events
- ✓ Browser APIs: window, localStorage
- ✗ Its JavaScript is sent to the browser

Default is the server. You opt in to the browser only when you need it:

```
"use client";
```

When does a component need "use client"?



Push the boundary down

Make the small interactive piece (a button, a search box) a Client Component, not the whole page. Only these islands ship JavaScript to the browser.

Next.js optimizes the boring but important parts



next/image

Resizes, compresses and lazy-loads images. Reserves space so nothing jumps.



next/font

Self-hosts fonts. No extra request to font servers and no layout shift.



next/script

Controls when third-party scripts load so they do not block the page.



Code splitting

Each route loads only its own JavaScript. Chunks for visible links are prefetched.

```
<Image src="/hero.png" alt="Hero"
width={800} height={400} />
```

```
const inter = Inter({
  subsets: ["latin"] });
```

```
<Script src="/analytics.js"
strategy="lazyOnload" />
```

Built in. You mostly just use the component.

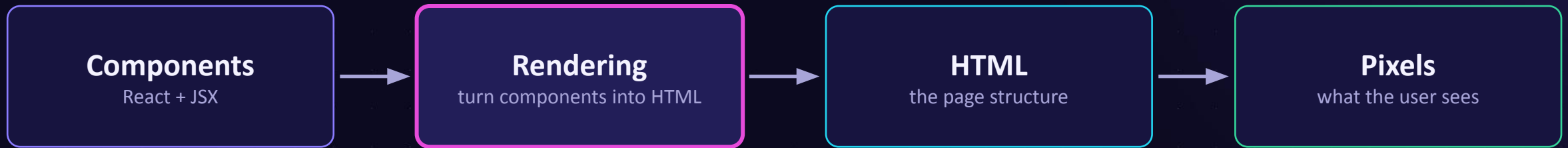
PART 2

Where and when does your UI become HTML?

Rendering, data fetching and caching are one connected decision.



What does "rendering" actually mean?



WHERE does it happen?

On the server

In the browser



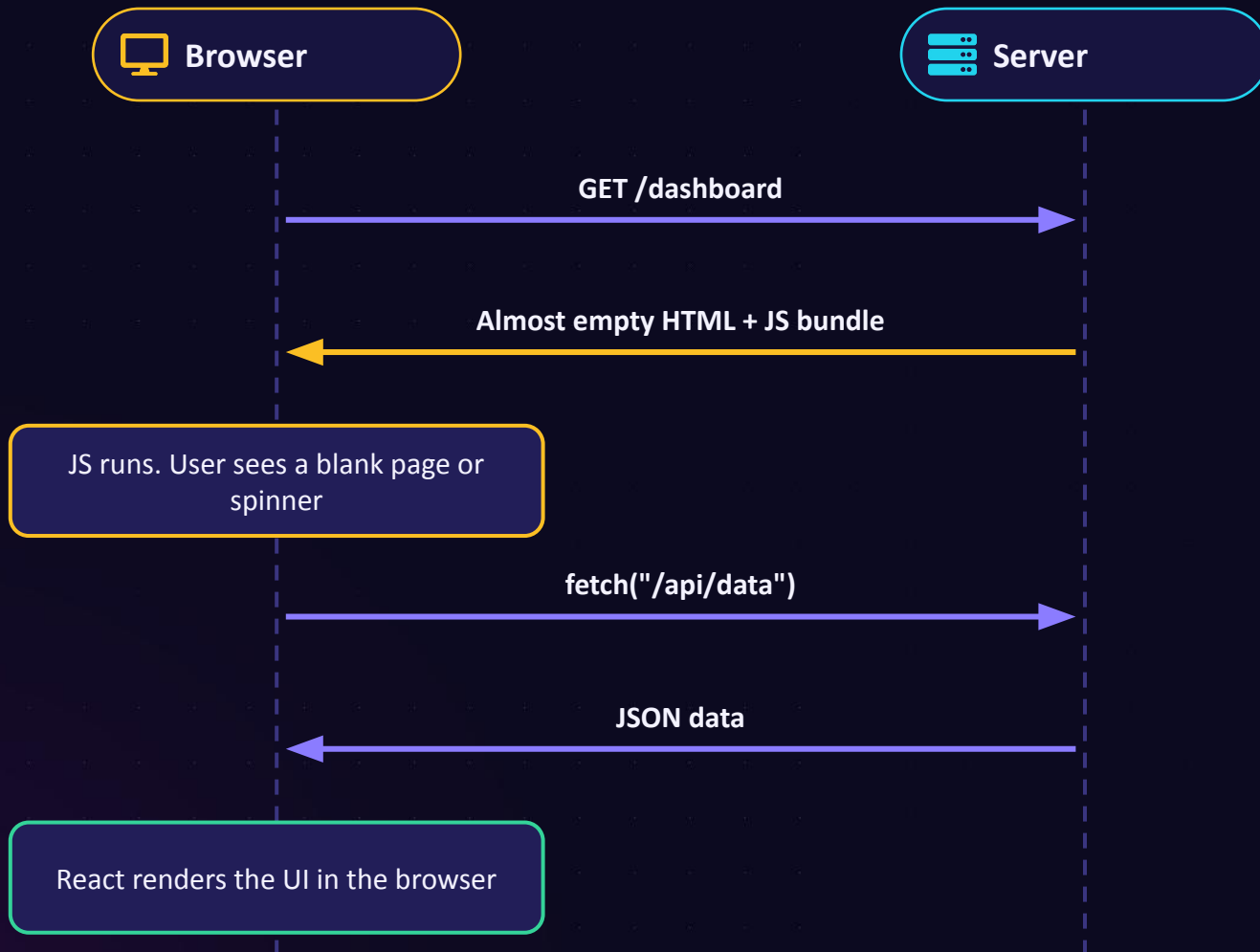
WHEN does it happen?

At build time

At request time, or on a schedule

CSR, SSR, SSG and ISR are just different answers to WHERE and WHEN.

Client-Side Rendering: the browser does the work

**WHERE**

In the browser

WHEN

After the page loads, on every visit

GOOD FOR

Private dashboards, highly interactive UI

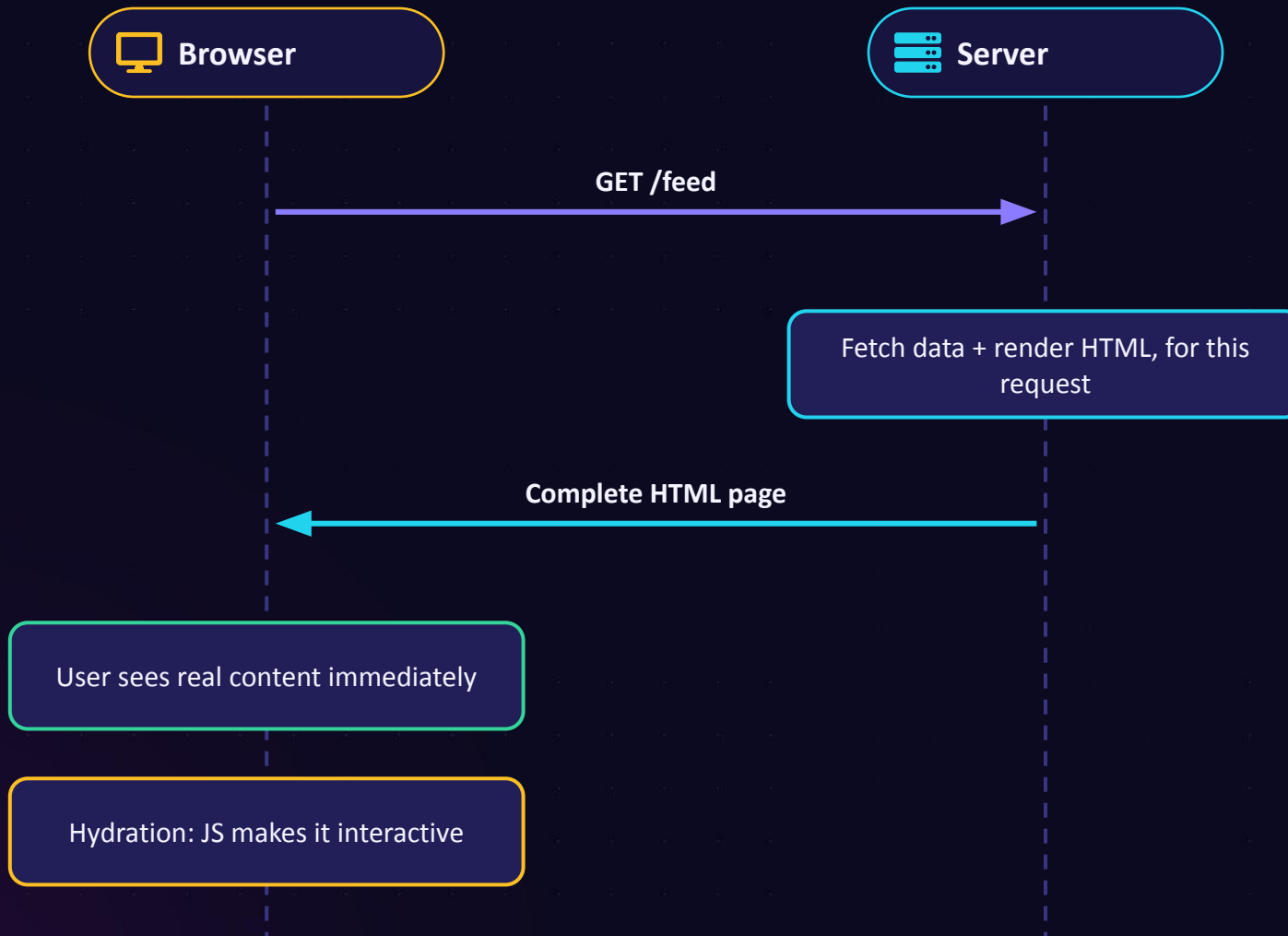
TRADE-OFF

Blank first paint, more JS, weak SEO

IN NEXT.JS

Client Components fetching in the browser

Server-Side Rendering: fresh HTML per request



WHERE

On the server

WHEN

At request time, every request

GOOD FOR

Personalized or fast-changing pages, SEO

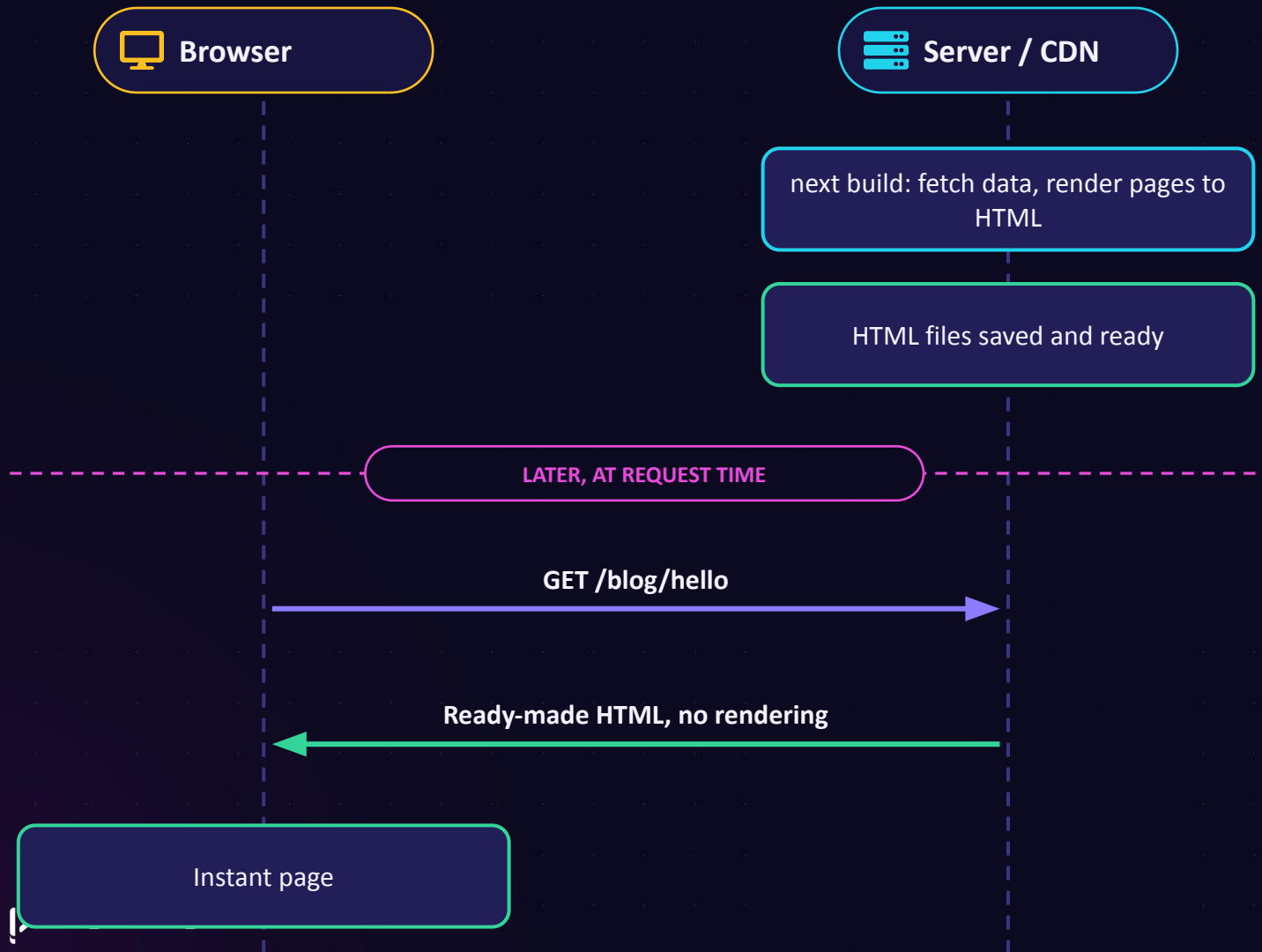
TRADE-OFF

Server works on every visit, slower than static

IN NEXT.JS

Dynamic rendering (uncached data or per-request info)

Static Site Generation: build once, serve fast



WHERE

On the server

WHEN

At build time, once

GOOD FOR

Blogs, docs, marketing pages

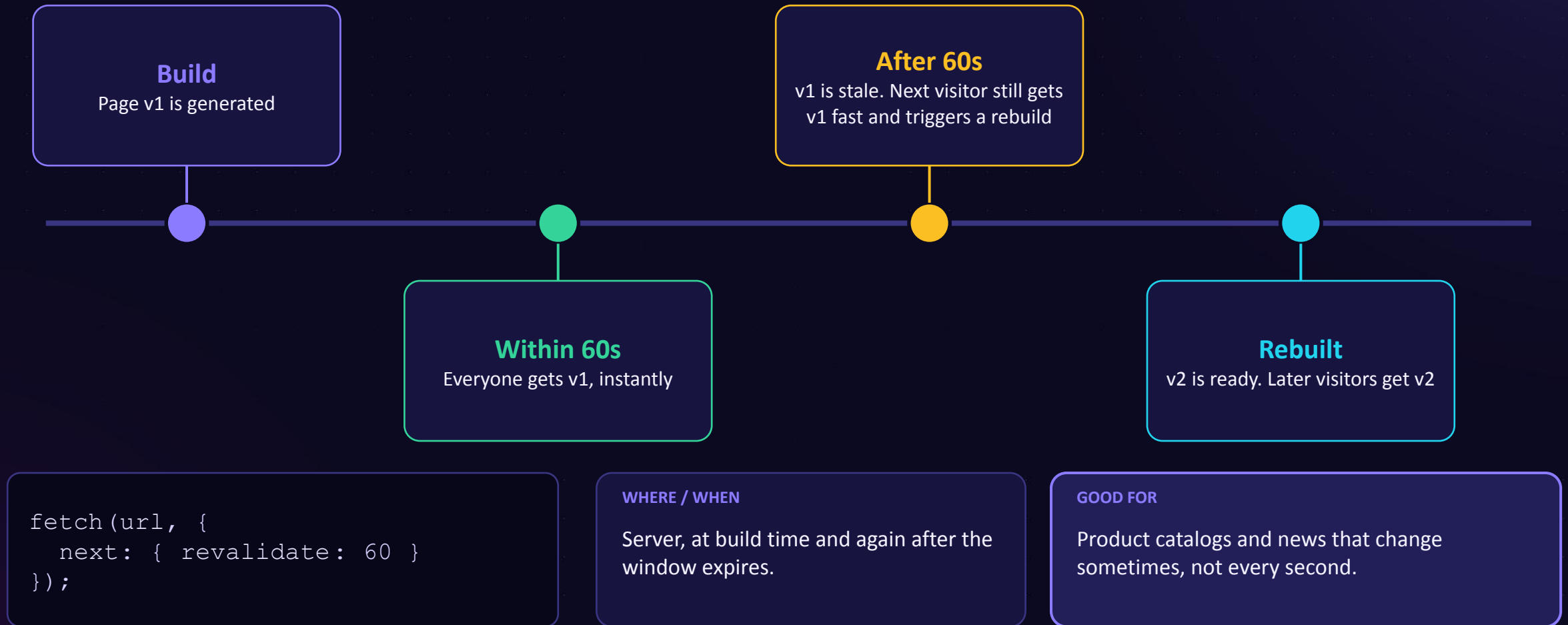
TRADE-OFF

Content is only as fresh as the last build

IN NEXT.JS

The default for pages with cacheable data

Incremental Static Regeneration: static that refreshes



The rendering map: where × when



Earlier is faster, later is fresher. Ask: how often does the data change? Is it per user?

Fetch where the data lives: on the server

Server-side fetching

the default choice

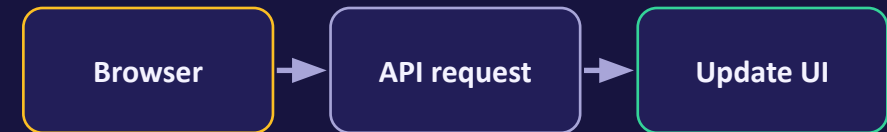


Close to the data. Secrets stay private. Less JS. HTML arrives with data inside.

```
export default async function Products() {
  const res = await fetch(API_URL);
  const products = await res.json();
  return <ProductList items={products} />;
}
```

Client-side fetching

when it makes sense



- ✓ The user types, clicks or scrolls
- ✓ Data must update live
- ✓ It depends on browser-only state

```
"use client";
useEffect(() => {
  fetch(`/api/search?q=${q}`)
    .then((r) => r.json()).then(setResults);
}, [q]);
```

Data fetching decides how a page renders

HOW DOES THE DATA BEHAVE?

Rarely changes

docs, blog posts

Changes every few minutes

catalog, news

Different on every request

per user, real time

Depends on what the user does

search, live widgets

HOW YOU ASK FOR IT

```
fetch(url, { cache: "force-cache" })
```

```
next: { revalidate: 60 }
```

```
fetch(url, { cache: "no-store" })
```

```
useEffect(() => fetch(...))
```

HOW THE PAGE RENDERS

Static (SSG)

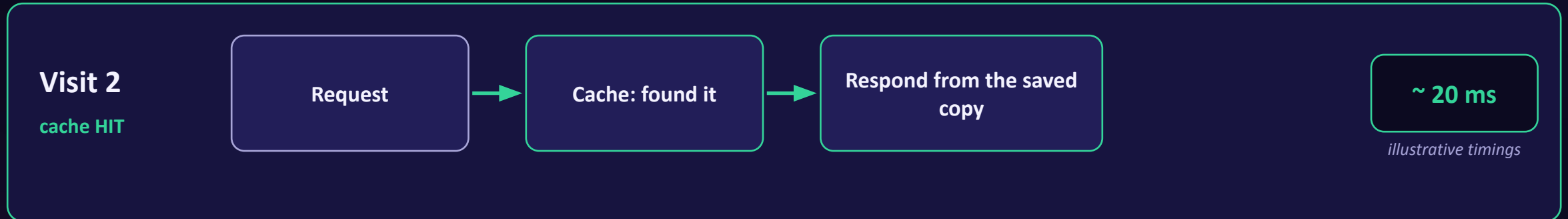
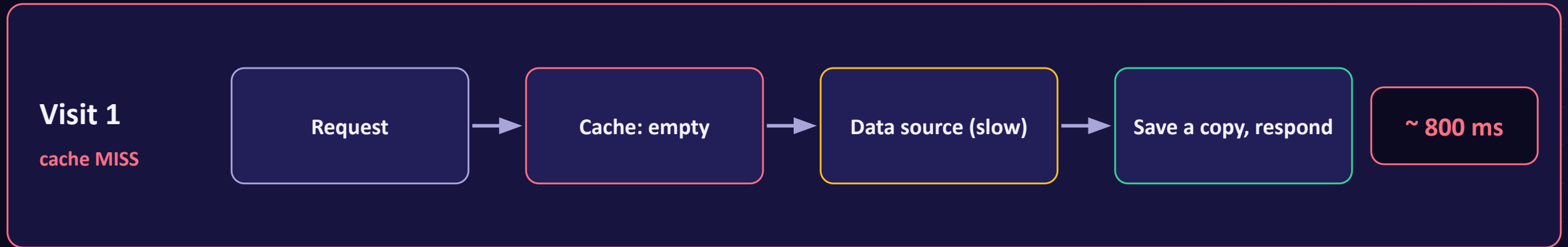
Static + refresh (ISR)

Dynamic (SSR)

Client-side (CSR)

Same page, different data needs, different rendering.

Caching: do expensive work once, reuse the result



Faster pages



Less work for the server



Fewer calls to slow APIs

Caching lives at several levels



In the browser

Pages you already visited come back instantly.



Saved pages

Static and ISR pages are stored as ready HTML.

Rendering



Saved data

Results of fetch calls are stored and reused.

Data fetching



Database / API

The source of truth. Also the slowest stop.

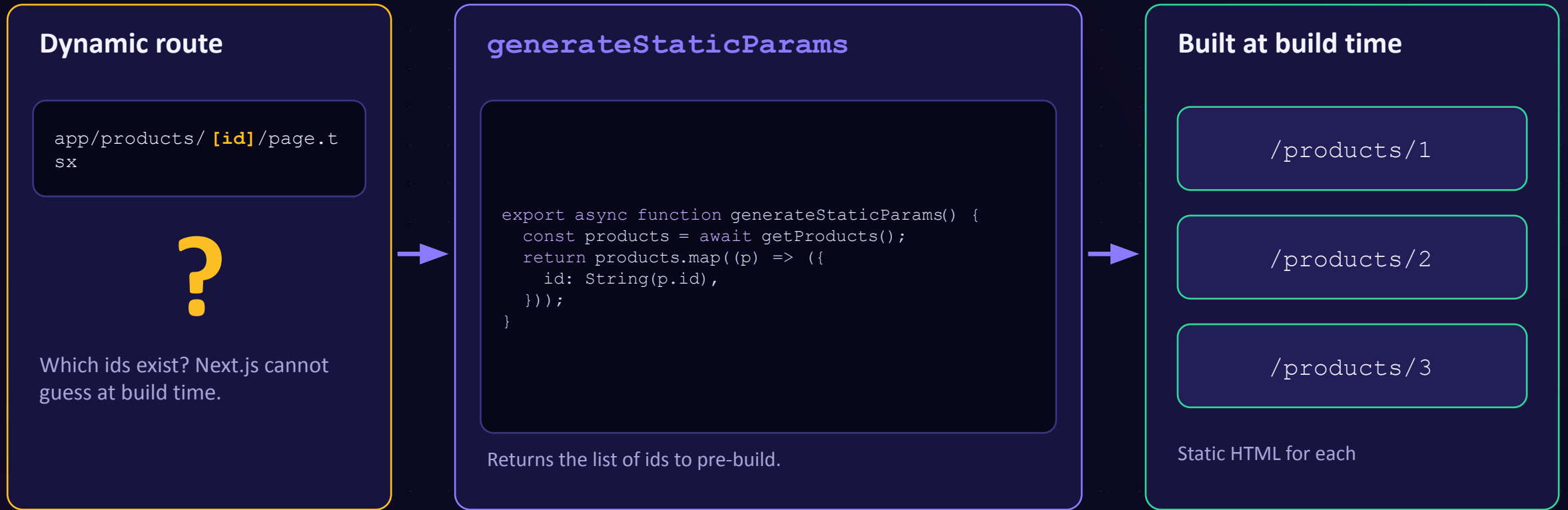
Deeper request
= slower response

A HIT STOPS EARLY

The higher the layer that answers, the faster the user sees the page.

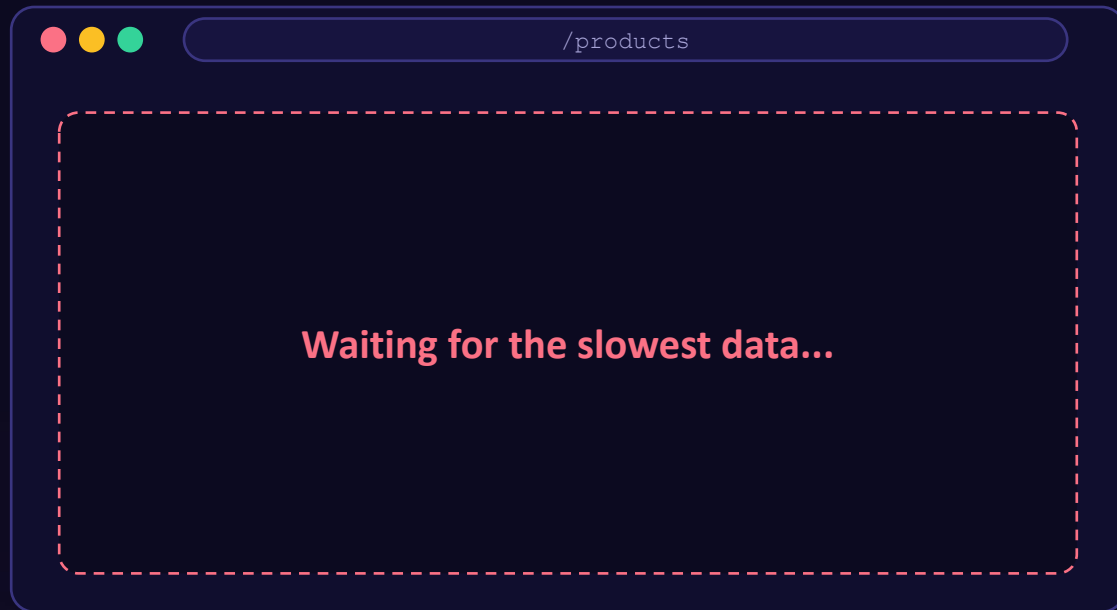
```
revalidate: 60
```

Dynamic routes + static generation



Not in the list? It is rendered on the first request, then saved for next time.

Don't make users wait for the slowest part



Without Suspense: the whole page waits



loading.tsx

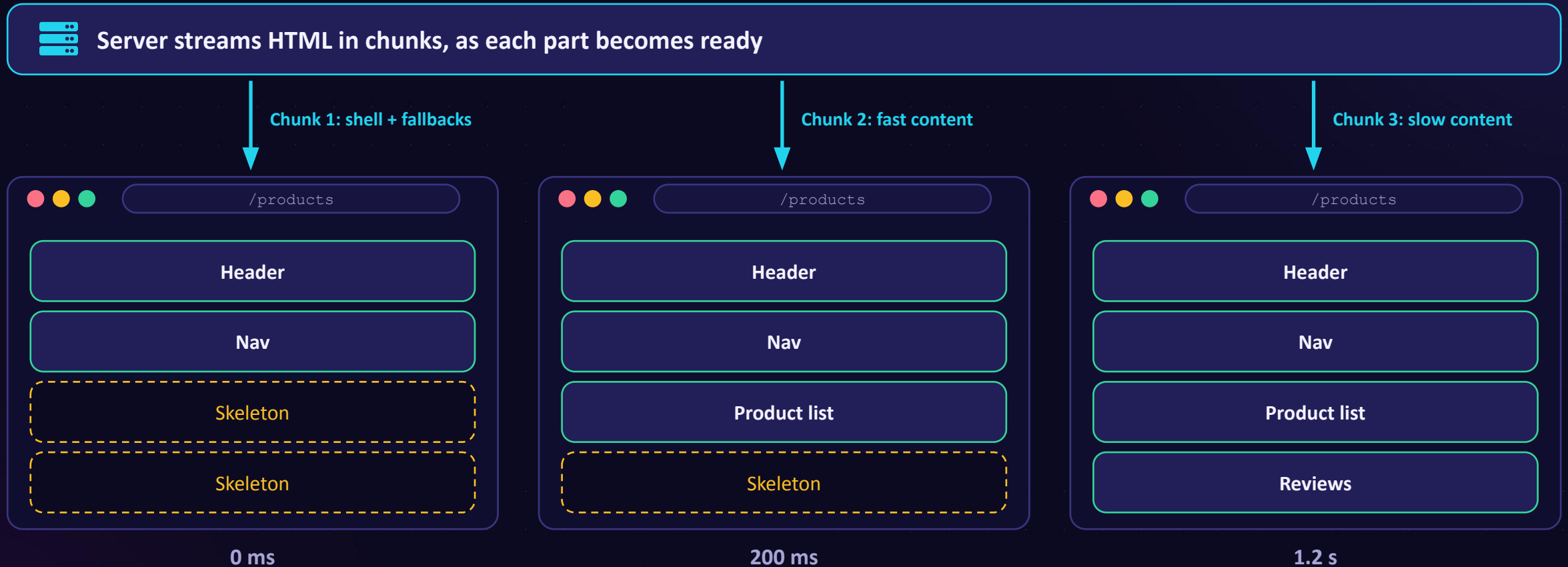
Instant loading UI for a whole route



With Suspense: show what is ready, fallback for the rest

```
<Suspense fallback={<Skeleton />}>  
  <SlowReviews />  
</Suspense>
```

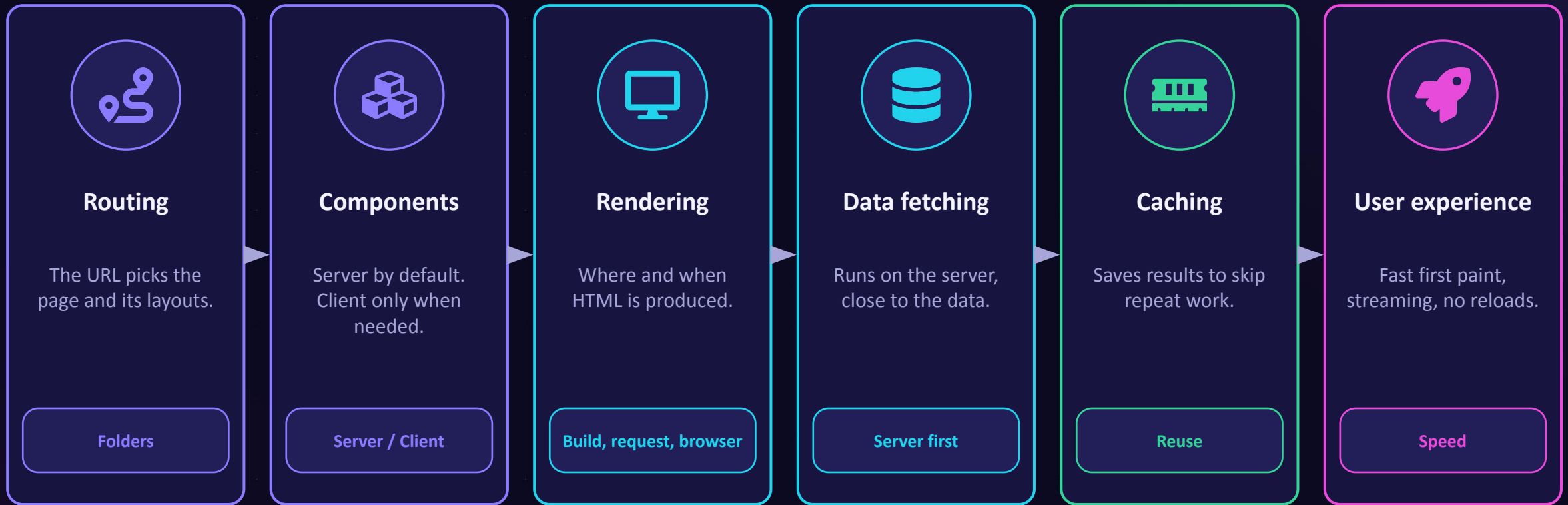
Streaming: the page fills in as data arrives



illustrative timings

Same data, sooner feedback. That is perceived performance.

How everything connects



Every choice changes the same thing: how fast users see something useful.

Follow one request: /products/42

1 Routing

/products/42 matches
app/products/[id]/page.tsx

2 Layouts

Root and products layouts
wrap the page

3 Components + data

A Server Component
fetches the product directly

4 Rendering

generateStaticParams
pre-built it. revalidate keeps
it fresh

5 Caching

The page and its data are
saved for the next visitor

6 Streaming

Slow reviews stream in
behind a Suspense fallback

7 Client island

AddToCartButton uses "use
client" to handle clicks

Routing → Component → Rendering → Data → Caching → User experience

Questions to ask when building a feature

1

What URL is this?

A folder with `page.tsx`. Use `[id]` for dynamic values.

2

Does UI repeat across pages?

Put it in a `layout.tsx` so it persists.

3

Needs state, events or browser APIs?

Yes: "use client". No: stay on the server.

4

How often does the data change?

Rarely: static. Sometimes: revalidate. Always: dynamic.

5

Which part is slow?

Wrap it in `Suspense` and stream it.

6

Dynamic route with known pages?

Add `generateStaticParams` to pre-build them.



Self-check: can you explain how routing, rendering, data and caching connect?



ONE CONNECTED SYSTEM

Route it. Componentize it.
Render it. Feed it.
Cache it. Stream it.



Now go build something and ask: where, when, why, and how does it connect?